Primera entrega del proyecto

Javier Felipe Contreras Díaz
2535936
contreras.javier@correounivalle.edu.co

Jhoan Esteban Penagos Lizcano
2517537
Jhoan.penagos@correounivalle.edu.co

Maria Camila Zuleta Peña
2540542
mariacamilazuletapena@gmail.com

Facultad de Ingeniería, Universidad del Valle
750015C: Fundamentos de Programación Orientada a Objetos
Victor Andres Bucheli Guerrero
Junio de 2026

Para la realización del proyecto, en primer lugar es necesario realizar un diagnostico del requerimiento, es decir evaluar qué es lo que se está pidiendo y evaluarlo como un objetivo con resultados esperados.

En este caso se pide un tablero (grid) donde diversos buses de colores están bloqueados por el trafico. Acá ya hemos identificado 2 clases, una clase **Tablero** y otra clase **Vehículo**. Adicional a esto, habrán pasajeros que abordarán los distintos vehículos, razón por la cual habrá otra clase **Pasajero.** Teniendo en cuenta el modelo MVC, habrán al menos otra 3 clases, el model, el view y el controller.

Identificador	Requerimiento Funcional	Descripción Detallada	Resultado Esperado
RF1	**Carga Dinámica de Niveles .txt**	El sistema debe leer archivos planos (nivel1.txt, nivel2.txt, etc.) usando std::ifstream y procesar las dimensiones del tablero, la flota de vehículos y la cola de pasajeros.	Matriz de juego inicializada correctamente en memoria sin desbordamientos.
RF2	**Inicialización del Escenario (Tablero)**	Ubicar dinámicamente cada vehículo (Vehiculo*) en la matriz bidimensional zonaCoches respetando su posición inicial (X, Y), tamaño y orientación.	El mapa se despliega en consola mostrando las posiciones exactas de los objetos y los obstáculos.
RF3	**Validación de Colisiones y Límites**	Verificar antes de cada movimiento que las celdas de destino estén vacías (nullptr) y dentro de los rangos válidos de la matriz	Bloqueo del movimiento si el coche choca contra otro o contra la pared.
RF4	**Desplazamiento**	Permitir al usuario seleccionar un vehículo por sus coordenadas y moverlo exclusivamente en la dirección de su orientación lineal asignada.	Modificación exitosa de las variables posX y posY del objeto y su debida impresión
RF5	**Abordaje por similitud de Color**	Procesar la cola de peatones (filaPasajeros). Si el color del primer pasajero coincide con el del vehículo estacionado en la zonaRecogida, este aborda liberando el espacio en la fila.	El pasajero pasa de la fila general al vector sillas del vehículo.
RF6	**Detección de Estado de Derrota**	Evaluar si la zonaRecogida excede la capacidad máxima de estacionamiento (4 vehículos) o si se encuentra completamente bloqueada sin movimientos posibles.	Terminación del juego -Game Over-

Según el resultado funcional que estemos buscando. Se tendrá un o una serie de clases y métodos para llevarlo a cabo.

Se tendrá una clase Tablero, en la cual se cargarán los objetos de las clases Pasajero y Vehículo. Estos estarán ligados a una lógica de la clase Model que tendrá funciones de carga de archivos txt, carga de objetos y parametros. Mientras que una clase controller manejará cierta lógicas de juego e imprimirá con ayuda de una clase View. Para que finalmente el main pueda organizar el flujo del programa.

Este será el diagrama de clases:

@startuml

class Controller {
- nivel: int
- vista: View&
- model: Model&
+ Controller(Model& auxModelo, View& auxVista)
+ iniciarJuego(): bool
+ movimientosVehiculos(): void
+ mostrarMapa(): void
+ pushPasajerosBus(): void
+ verificarZonaRecogida(): void
+ verificarVictoria(): bool
+ verificarDerrota(): bool
}

class Model {
- vehiculos: vector<Vehiculo>
- pasajeros: vector<Pasajero>
- tablero: Tablero
+ Model()
+ cargarTxt(int nivel): void
+ cargarVehiculosEnTablero(): void
+ cargarPasajerosEnTablero(): void
+ verificarColision(int posX, int posY): bool
+ verificarDireccion(char direccion, int posX, int posY): bool
+ verificarInputDireccion(): char
+ verificarPosicionVehiculo(): tuple<int,int>
+ verificarZonaRecogida(): void
+ moverVehiculo(int posX, int posY, int tam, char direccion): bool
+ pushPasajerosBus(int i): void
+ getVerificarDerrota(): bool
+ getTablero(): Tablero&
}

class View {
+ View()
+ mostrarMenu(): void
+ mostrarNivel(int nivel): void
+ mostrarMapa(Tablero& paradero): void
}

class Tablero {
- filas: int
- columnas: int
- zonaCoches: vector<vector<Vehiculo*>>
- filaPasajeros: vector<Pasajero*>
- zonaRecogida: vector<Vehiculo*>
- zonaVictoria: vector<Vehiculo*>
- verificarDerrota: bool
+ Tablero()
+ Tablero(int auxFilas, int auxColumnas)
+ setZonaCoches(Vehiculo* auxVehiculo): void
+ pushFilaPasajeros(Pasajero* peaton): void
+ verificarColision(int posX, int posY): bool
+ verificarDireccion(char direccion, int posX, int posY): bool
+ verificarDerrota(): bool
+ moverVehiculo(int posX, int posY, int tam): void
+ pushPasajerosBus(Pasajero* peaton, int i): void
+ verificarZonaRecogida(): void
+ pushZonaVictoria(Vehiculo* carro): void
+ getFilas(): int
+ getColumnas(): int
+ getFilaPasajeros(): vector<Pasajero*>&
+ getZonaRecogida(): vector<Vehiculo*>&
+ getZonaCoches(): vector<vector<Vehiculo*>>&
}
class Vehiculo {
- tipo: char
- color: char
- orientacion: char
- tamano: int
- posX: int
- posY: int
- sillas: vector<Pasajero*>
+ Vehiculo(char auxTipo, char auxColor, char auxOrientacion, int auxTamano, int auxPosX, int auxPosY)
+ getColor(): char
+ getTipo(): char
+ getTamano(): int
+ getOrientacion(): char
+ getPosX(): int
+ getPosY(): int
+ getSizeSillas(): int
+ setPos(int x, int y): void
+ pushPasajerosBus(Pasajero* peaton): void
}

class Pasajero {
- color: char

```
+ Pasajero(char auxColor)
+ getColor(): char
}


' Relaciones
Controller --> Model
Controller --> View
Model *-- Tablero
Model *-- Vehiculo
Model *-- Pasajero
Tablero o-- Vehiculo
Tablero o-- Pasajero
Vehiculo o-- Pasajero

@endum
```